

MONQ infrastructure documentation

November 29, 2014

0.1 Introduction

The infrastructure has been prepared to be within the same directory. This allows easily copying the infrastructure to different machines.

The infrastructure folder is composed of:

- automata - dictionary files in MWT format
- bin - configuration scripts to run the infrastructure
- config - infrastructure services and pipelines configuration
- doc - infrastructure documentation
- lib - Java jar libraries
- logging - logging folder for the infrastructure services
- medline - MEDLINE baseline and Lucene index
- programs - additional programs, including Apache Tomcat to run front end applications such as Whatizit
- xslt - XSLT files used to render XML files produced by the infrastructure into HTML. Front end services use them.

0.2 Setup

The infrastructure folder contains a bin folder with several scripts needed to run the infrastructure. The *setup.sh* can be used to set the environment variables needed by the infrastructure scripts. The MONQ environment variable needs to be adapted to the location of the infrastructure.

The script can be used to source its content into the current shell using the `.` (dot) before script as shown below (run from the bin folder location). Once the current shell is ready to run the infrastructure, all the other scripts can be used.

```
$ . setup.sh
```

0.3 Information extraction infrastructure

The infrastructure allows to set up text mining pipelines by concatenating several services. The services run as servers that can be run on different machines. Their configuration file, specifies the machine in which they run and the port number to contact them. The command to be run when starting them is setup as well. Configuration files are available from the config folder and use the extension `svr`.

```

<svr>
  <synopsis>...</synopsis>
  <environment>
  </environment>
  <access>public</access>
  <host>dell</host>
  <port>8003</port>
  <cmd>DictFilter ... </cmd>
</svr>

```

A server can be started and stopped using the same call, which acts as a switch.

```
$ startServer swissprot
```

Services write to a log file in the logging directory. A log file can be used to manage the status of the service and to identify any problems.

A service can be run using the DistFilter command. Text is provided in the standard input and the standard output shows the result of the processing by the services.

```
$ echo "<text>BRCA1</text>" | DistFilter svr=sentenciser
```

The output from the previous command is:

```

<text><SENT sid="0" pm="."><plain>BRCA1 </plain></SENT>
</text>

```

Pipelines can be built dynamically using the DistFilter command by concatenating several services as shown below:

```
$ echo "<text>BRCA1</text>" | DistFilter svr=sentenciser
svr=swissprot svr=bncfilter
```

```

<text><SENT sid="0" pm="."><plain><z:uniprot fb="0" ids="P38398,
Q6J6I8,Q6J6I9,Q6J6J0,Q864U1,Q95153,Q9GKK8">BRCA1</z:uniprot>
</plain></SENT></text>

```

Pipeline configurations can be prepared to be run more than once. Files with extension pipe (e.g. whatizitSwissprot.pipe) in the config folder are used to define pipelines.

```

<pipe>
  <desc>...</desc>
  <access>public</access>
  <dfas>
    <svr>sentenciser</svr>
    <svr>swissprot</svr>

```

```

    <svr>bncfilter</svr>
  </dfas>
  <style>/home/monq/xslt/whatizit.xslt</style>
</pipe>

$ echo "<text>BRCA1</text>" | DistFilter pipe=whatizitSwissprot

```

The previous command produces the following output, which is the same as when defining dynamically the pipeline.

```

<text><SENT sid="0" pm="."><plain><z:uniprot fb="0" ids="P38398,
Q6J6I8,Q6J6I9,Q6J6J0,Q864U1,Q95153,Q9GKK8">BRCA1</z:uniprot>
</plain></SENT></text>

```

0.4 MEDLINE index

A MEDLINE index has been prepared and it allows testing the infrastructure and provide analysis on MEDLINE.

The index can be rebuilt and some scripts exist for this purpose. The indexing process is split into file pre-processing and indexing.

In the pre-processing step, the files are annotated with the citation position and SwissProt identifiers. The preprocessMEDLINE.sh script can be used to annotate one file. The preprocessed MEDLINE files should be stored in the ../medline/2014/baseline.annotated, which should be updated according to the location of the MEDLINE files. An example call is presented below. Processed files should be compressed using gzip.

```

$ cat ../medline/2014/baseline/medline14n0001.xml | \
sh preprocessMEDLINE.sh | gzip >
../medline/2014/baseline\_annotated/medline14n0001.xml.gz

```

Indexing is done by running the indexing.sh script. It will look for the files in the baseline_annotated folder. The script allows indexing four files at the same time. The index is stored in the index folder. Whatizit should be stopped while manipulating the index. Indexing could be done on a different folder to avoid stopping Whatizit.

Retrieval of MEDLINE citations based on the Lucene index is done by using the qbmarsdf service.

```

$ echo cancer | DistFilter svr=qbmarsdf

```

0.5 Whatizit

Whatizit runs on Apache Tomcat. It is preinstalled and can be run using the following script. By starting the Tomcat, Whatizit is started.

```
$ cd $MONQ/programs/apache-tomcat-7.0.52/bin
$ sh startup.sh
```

To stop it the shutdown.sh script needs to be run.

```
$ cd $MONQ/programs/apache-tomcat-7.0.52/bin
$ sh shutdown.sh
```

0.6 Future work